GPU Architecture meeting 4

BE BOUNDLESS

Thank you everyone!

Final meeting for the quarter!

- > **Github commits**
- > **Testing your modules**
- > **Where we're going**
- > **Next steps**

Github

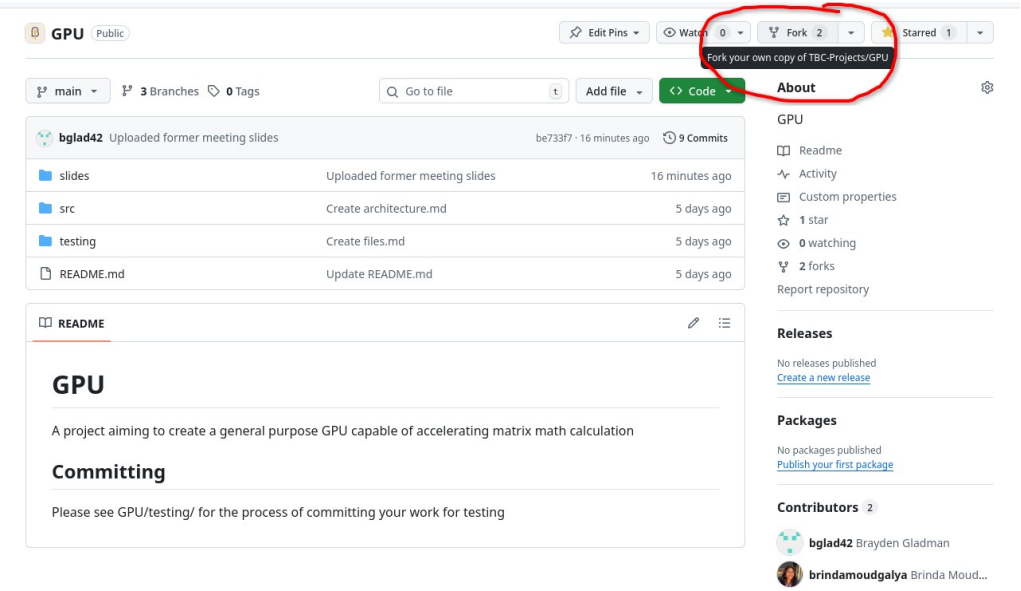
Please do the following

> **If you haven't yet, create a GitHub Account**

> **Access the GPU repository**

> **Clone it**

> **https://github.com/TBC-Projects/GPU**

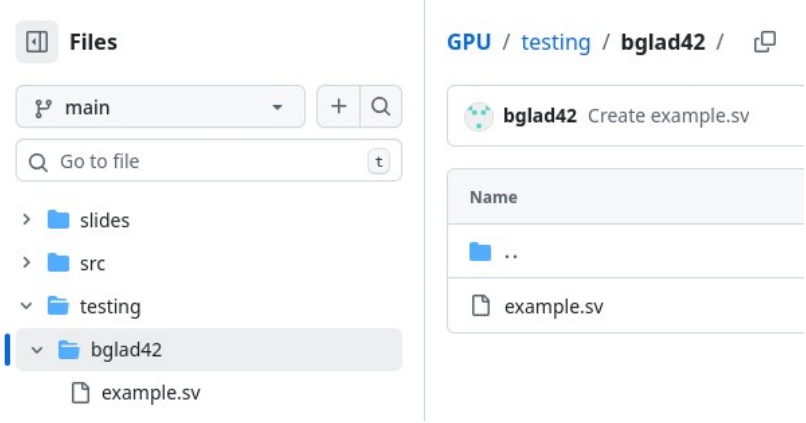


Github

Now I need you to commit any modules you've made

Hierarchy should be as follows

> **Gpu**
- ↳ **Testing**
  - ↳ **[Your Name/netid/username]**
    - **[example].sv**
    - **[example2].sv**



When your changes are made, make a pull request to the main branch!!

Logic Simulation

So yeah, it’s not immediately obvious if your solution is the correct one, we should test behavior

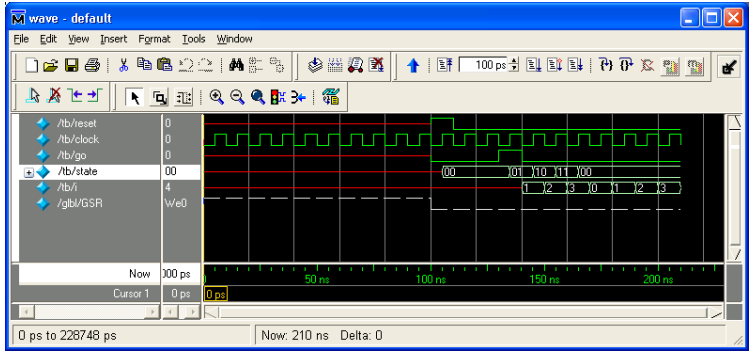
> **Simulate the device**

- **Device Under Test**

> **Stimulate the device**

- **Inputs**

Read outputs, match to correct



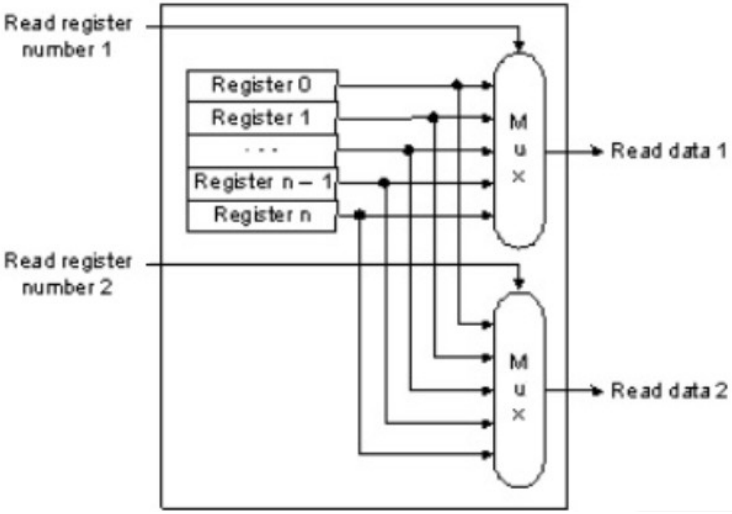
Simulation Demo

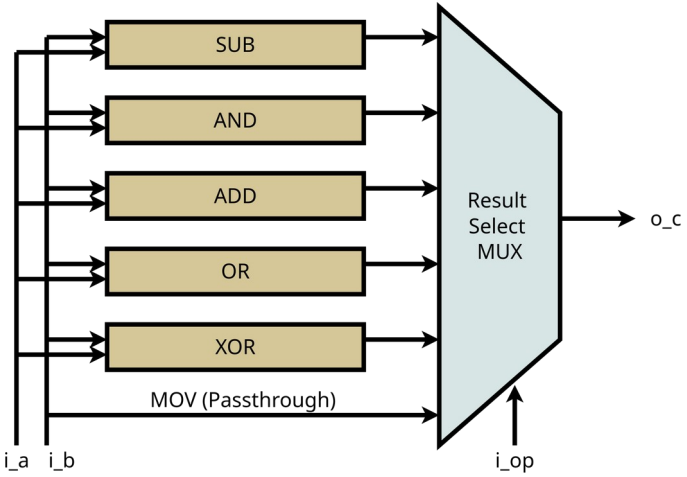
Basic test bench

> **4 to 1 mux**

Where this is all going

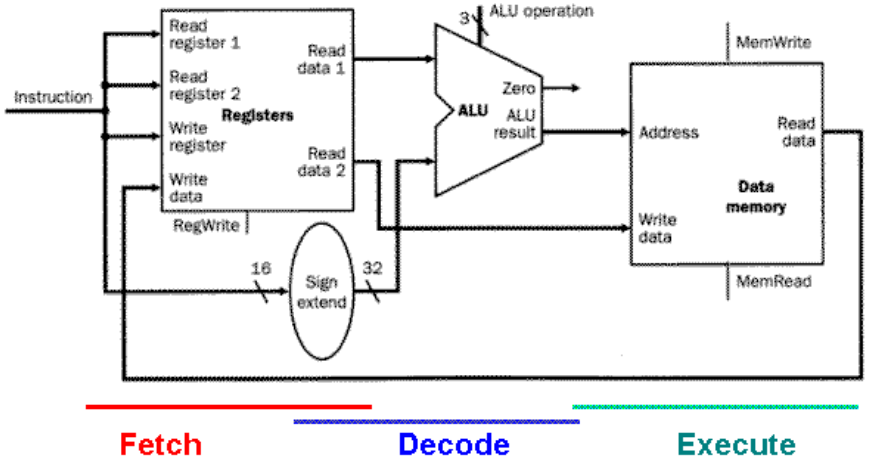
The Registers look like this, and feed their two outputs to the ALU to do one of its functions





Where this is all going

Once these pieces completed, we put together to make a processor (right now, we are hand waving memory)



Henceforth this = Thread

Where this is all going

So when does this become a gpu?

> **What if we had a bunch of these?**

Where this is all going

Our registers will contain this

X15: Read only identifier
X14: Block dimension
X13: Read only 0
...
X0: General Purpose Register

- **Identifier (we will need this)**
- **Important for matrices**
- **Zero only (critical for a number of operations)**

- **Rest are for storing the actual values**

Where this is all going

Zooming out

X15: Read only identifier
X14: Block dimension
X13: Current Block
...
X0: General Purpose Register

- **Identifier (we will need this)**
- **Information about current process**
- **Information about current process**

- **Rest are for storing the actual values**

Where this is all going

Zooming out further

- **Scheduler to manage what each thread does**
- **All of this together will be a <u>core</u>**
- **We can go further**

<table>
<tr><td colspan="2">Scheduler</td></tr>
<tr><td colspan="2">Controls</td></tr>
<tr><td>Thread 0</td><td>Thread 1</td></tr>
<tr><td>Thread 2</td><td>Thread 3</td></tr>
</table>

Where this is all going

Zooming out even further

> **Dispatch breaks up an input kernel into blocks, distributes it to an available core, tracks state of each core**

Dispatch	
Core 0	Core 2
Core 2	Core 3

Once again, thank you all!

Next meeting in the spring!